

CS 452/652 Winter 2025 - Lecture 2

Devices and Trains

Jan 9, 2025 [prev](#) [next](#)

BCM - [Broadcom BCM2711 data sheet](#)

Memory

- see BCM Section 1.2.1, Figure 1 “Low Peripheral”
- main memory: assume 0x00000000 - 0x3FFFFFFF (1 GB)
- top 76MB used for VC ([config.txt](#))
- memory-mapped device registers: "magic" addresses
 - see BCM Section 1.2.4: base address 0x7E000000 → 0xFE000000

Timer

- no time of day needed, no remote synchronization (very hard problem)
- ARM timer (BCM Chap 10) vs. System timer (BCM Chap 12)
- ARM timer follows CPU-frequency → use system timer
- system timer
 - free-running counter at stable frequency: 1 MHz
 - 7 control registers starting at offset 0x203000
 - CLo/CHi → timer counter
 - C0-3: compare and set status in CS
 - query timer and/or wait for interrupt via compare registers

General-Purpose Input/Output (GPIO)

- configurable hardware interface
- BCM Chap 5, Page 78, registers at offset 0x00200000
- iotest contains code for setting up GPIO
- no reason to mess with this

Universal Asynchronous Receiver/Transmitter (UART)

- serial (character) device: 1 byte at a time
- BCM Chap 11 (not Chap 2): UART0 and UART3
 - device registers at offset 0x201000 (UART0) and 0x201600 (UART3)
 - data register (DR) at offset 0x00
 - FIFO buffer (optional, default size 1)
- configuration: speed, byte representation, FIFO
 - Channel 1 (terminal): 115200 baud (8 bits, no parity, 1 stop bit)
 - Channel 2 (train): 2400 baud (8 bits, no parity, 2 stop bits)
- output (transmit, TX)
 - write to DR
 - UART moves to FIFO
 - FIFO entries sent via serial line
- input (receive, RX)

- reception from serial line into FIFO
- first element in FIFO is always available in DR
- read from DR
- flag register (FR) describes status of TX/RX buffers (full/empty)
 - check before send/receive
- timing
 - receive: need to be fast enough
 - transmit: paced according to line speed when checking TX state in FR
 - how long does it take to transmit a byte to the console? to the Marklin controller?
 - sender speed vs. serial line speed vs. receiver speed
 - later: flow control support
 - for A0: manually wait (~20ms for train) after each train command

Trains & Track

- 1- or 2-byte commands: operation (, address)
- trains:
 - 0-14 speed level
 - 15 reverse direction
 - add 16 to turn on headlights
- trains slow down gradually
 - do not directly reverse a rolling train
- switches:
 - 0x21 straight, 0x22 turn (curved)
 - changes a switch setting activates a solenoid
 - deactivate at most 250ms after sending the switch command
 - deactivate by:
 - sending another switch command (to any switch)
 - send the special solenoid off command (0x20)
 - single byte - turns off solenoid from last switch
 - middle "double" switches
 - avoid C/C state (and S/S is not very helpful either)
 - if curved left (CS), and want curved right (SC)
 - first set left switch to S (SS)
 - then set right switch to C (SC)
- sensors:
 - 5 banks of 16 up to 16 sensors
 - banks labeled as A, B, C, D, E on track
 - individual sensors labeled A13, A14
 - sensors are directional: A15 →, A16 ←
 - when trains trigger sensor, changes sensor state in Marklin controller from 0 to 1
 - read the state of all sensors by sending command 0x85 (0x80 + 0x05)
 - means read state of all banks up to and including 5th banks
 - then read 10 bytes of data from the Marklin
 - two bytes (16 bits) per bank
 - or read individual banks (0xC0 + bank number)
 - controller is in one of two sensor reset modes: *on* or *off*
 - if reset mode is *on*, the sensor bits are cleared each time they are read
 - if reset mode is *off*, reading the sensor bits does not clear them
 - The single-byte command 0xC0 can be used to turn the reset mode to *on* (0x80 turns *off*).

Notes

- turn on the headlights! (verify communication and direction)
- can use extra serial cable to connect gtkterm to RPi Channel 2 for debugging
- IMPORTANT: turn off the last solenoid using 0x20!
- IMPORTANT: track communication is half-duplex, i.e., no control during sensor reporting

Polling Loop

- polling $\neq$ busy-wait

```
for (;;) {  
    if (c1) a1();  
    if (c2) a2();  
    ...  
}
```

vs.

```
for (;;) {  
    while (!c1);  
    a1();  
    while (!c2);  
    a2();  
    ...  
}
```